

异常检测（三）：评估、阈值与代价

Evaluation, Threshold Selection, and Cost-sensitive Decisions



基于李宏毅老师《机器学习 2025》课程整理

2026 年 6 月 8 日

视频信息

频道：李宏毅深度学习课堂 (Bilibili)

时长：约 14 分 04 秒

链接：<https://www.bilibili.com/video/BV1TAtwzTE1S?p=31>

目录

1	从 confidence baseline 到完整评估流程	3
1.1	训练标签与评估标签不一样	3
1.2	从 development set 到 testing set	4
1.3	本章小结	5
2	观察 confidence 分布：阈值为什么难选	5
2.1	理想情形与现实情形	6
2.2	阈值移动带来的偏好变化	6
2.3	本章小结	6
3	Accuracy 的陷阱与两类错误	6
3.1	高 accuracy 可能代表系统什么都没做	7
3.2	两种错误：false alarm 与 missing	7
3.3	用混淆矩阵看错误结构	8
3.4	本章小结	8
4	用 cost table 决定系统偏好	9
4.1	两个阈值，对应两种系统行为	9
4.2	Cost Table A：误报很贵	10
4.3	Cost Table B：漏报很贵	10
4.4	医学筛查中的直觉	11
4.5	本章小结	11
5	不固定阈值的评价：ranking 与 AUC	12
5.1	从二元决策到排序	12
5.2	ROC 与 AUC 的含义	12
5.3	AUC 能回答什么，不能回答什么	13
5.4	本章小结	13
6	总结与延伸	13

1 从 confidence baseline 到完整评估流程

上一讲已经说明：在有标注的异常检测设定中，可以先训练一个普通分类器，再把分类器给出的 confidence score 当成异常检测分数。本讲继续往下问一个更实际的问题：既然系统已经可以输出 normal 或 anomaly，那么我们如何判断它做得好不好？又该如何决定阈值 λ ？

课程仍用辛普森家族人物分类作例子。训练资料是辛普森家族角色图片，每一张图片 x 都有角色标签 $\hat{y}$ ，例如它到底是 Homer、Marge、Bart，还是其他辛普森角色。分类器训练完成后，可以对输入图片输出类别预测与 confidence score $c(x)$ 。异常检测规则写成

$$f(x) = \begin{cases} \text{normal}, & c(x) > \lambda, \\ \text{anomaly}, & c(x) \leq \lambda. \end{cases}$$

其中 λ 是阈值。阈值高，系统会更谨慎，更容易把样本拒绝为异常；阈值低，系统更宽松，更容易接受样本为正常。

Example Framework

Training Set: Images x of characters from Simpsons.

Each image x is labelled by its characters $\hat{y}$.

Train a classifier, and we can obtain confidence score $c(x)$ from the classifier.

$$f(x) = \begin{cases} \text{normal}, & c(x) > \lambda \\ \text{anomaly}, & c(x) \leq \lambda \end{cases}$$

Dev Set: Images x

Label each image x is from Simpsons or not.

Created with EverCam.
<http://www.camdemy.com>

图 1: 异常检测框架中的 development set: 它不再标角色身份，而是标注输入是否来自辛普森家族¹

1.1 训练标签与评估标签不一样

这个例子最容易混淆的地方在于：训练分类器时需要的标签，和评估异常检测器时需要的标签，并不是同一种标签。

训练分类器时，资料全部来自辛普森家族人物。模型需要知道每张图是哪一个角色，所以训练标签是多类别标签：

$$y \in \{\text{Homer}, \text{Marge}, \text{Bart}, \dots\}.$$

但到了异常检测阶段，我们真正关心的不是“它是哪一个辛普森角色”，而是“它是不是辛普森家族

¹ 视频画面时间区间：00:01:45–00:02:00。

人物”。因此 development set 的标签应该是二元标签：

$$z \in \{\text{from Simpsons, not from Simpsons}\}.$$

这件事很重要。假如 development set 仍然只包含辛普森角色身份标签，我们最多只能评估分类器在 closed-set classification 上分得准不准，却无法评估它遇到非辛普森图片时会不会拒绝。异常检测要模拟的是测试时可能出现的开放世界：输入可能来自训练分布，也可能来自训练分布之外。

development set 的角色

Development set 不是拿来重新训练分类器的主要资料，而是拿来调异常检测系统。它应该尽量模拟未来测试时的情境：里面既要有正常样本，也要有异常样本；标签要回答“是否来自正常分布”，而不是只回答“正常分布内部是哪一类”。

1.2 从 development set 到 testing set

有了 development set 之后，我们可以把完整系统 $f(x)$ 放到这个集合上计算表现。这里的完整系统包括分类器、confidence score 定义、阈值 λ ，以及任何额外的超参数。换句话说，development set 看的不是单独的分类准确率，而是整个异常检测流程在模拟测试环境中的效果。

Example Framework

Training Set: Images x of characters from Simpsons.

Each image x is labelled by its characters $\hat{y}$.

Train a classifier, and we can obtain confidence score $c(x)$ from the classifier.

$$f(x) = \begin{cases} \text{normal}, & c(x) > \lambda \\ \text{anomaly}, & c(x) \leq \lambda \end{cases}$$

Dev Set: Images x

Label each image x is from Simpsons or not.

We can compute the **performance** of $f(x)$

Using dev set to determine λ and other hyperparameters.

Testing Set: Images x $\longrightarrow$ from Simpsons or not

Created with EverCam.
<http://www.camdemy.com>

图 2: 用 development set 计算系统表现，并据此选择阈值 λ 与其他超参数；最终再放到 testing set 上评估²

流程可以整理成四步：

1. 用训练集训练一个分类器，使其能够区分辛普森家族内部的角色。
2. 从分类器输出中定义 confidence score，例如最大 softmax 分数或其他 confidence 估计。

²视频画面时间区间：00:03:00–00:03:15。

3. 在 development set 上调阈值 λ ，并比较不同系统设定的表现。
4. 固定所有选择之后，在 testing set 或真实线上资料中评估最终表现。

这个流程背后的原则和普通机器学习相同：不能一边看测试集结果一边改阈值，否则测试集就变成了 development set，最终数字会过于乐观。真正可信的测试表现，应该来自所有设计选择都固定之后的独立资料。

1.3 本章小结

异常检测系统不是只要会输出一个 confidence score 就结束了。训练集负责学会正常分布内部的类别结构，development set 负责模拟“正常/异常”二元判断并选择阈值，testing set 才负责评估最终系统。特别要记住：异常检测的 development label 应该回答“是否来自正常分布”，而不只是回答正常类别内部的身份。

2 观察 confidence 分布：阈值为什么难选

接下来课程用一个具体 development set 说明阈值选择为什么不只是形式问题。这个集合中有 100 张辛普森家族图片，也有 5 张不是辛普森家族的异常图片。分类器对每张图片都会给出 confidence score，系统再根据阈值决定 normal 或 anomaly。

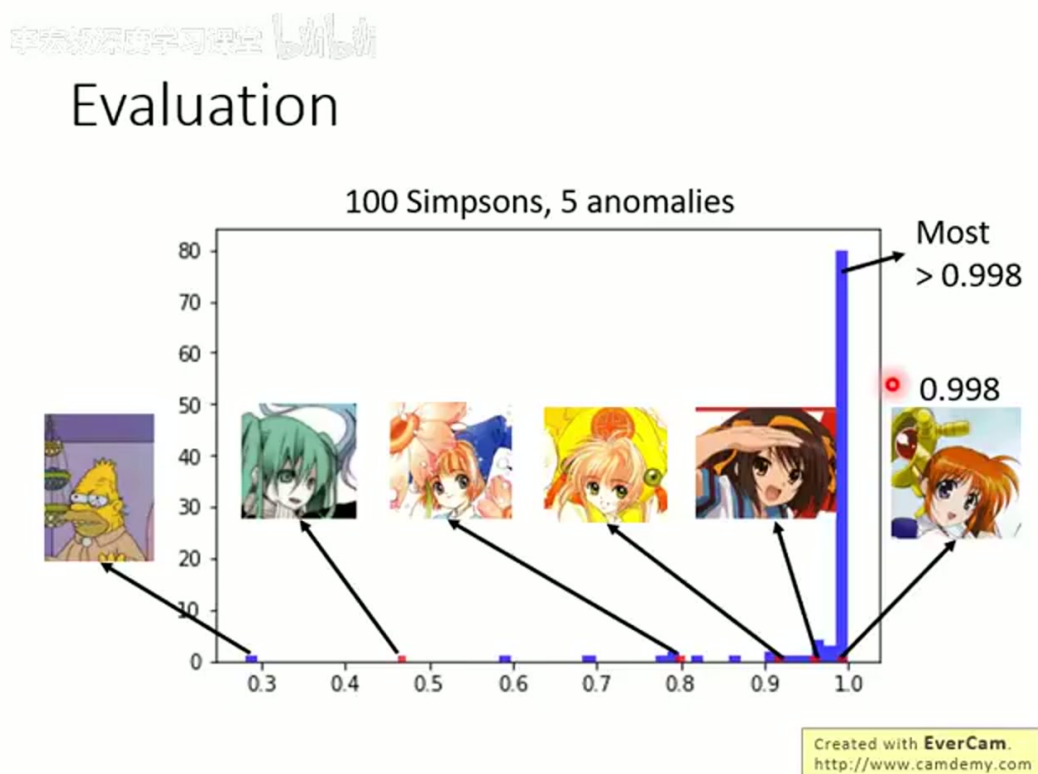


图 3: 100 张正常图片与 5 张异常图片的 confidence 分布：大多数正常样本分数高于 0.998，但异常样本也可能拿到不低的分数³

³ 视频画面时间区间：00:06:15–00:06:30。

2.1 理想情形与现实情形

如果 confidence score 完美，那么所有正常样本都会得到很高分，所有异常样本都会得到很低分。此时只要在两群分数之间放一个阈值，就能把 normal 和 anomaly 完全分开。但真实情形通常没有这么干净。

图中大多数辛普森家族图片的 confidence 都非常高，许多甚至高于 0.998。这说明分类器对训练分布内的角色相当有把握。可是，非辛普森图片并不一定都会得到低分。有些异常图片仍然可能被分类器看成某个已知角色的脸、颜色或局部形状，从而得到较高 confidence。

因此，confidence score 更像是一条有用但有噪声的线索，而不是一条绝对可靠的边界。阈值 λ 的作用，就是把这条连续分数转换成二元决策。转换之后，分数分布中任何重叠区域都会变成错误。

2.2 阈值移动带来的偏好变化

阈值 λ 越高，系统越严格：只有非常高 confidence 的样本才会被接受为 normal。这样做通常会减少漏掉异常的机会，但也会让部分正常样本被误判为 anomaly。阈值 λ 越低，系统越宽松：更多样本会被接受为 normal。这样做会减少正常样本被打扰的情况，但异常样本更容易混进来。

可以把阈值理解成系统的“保守程度”旋钮：

阈值设定	倾向	常见后果
较高的 λ	更容易拒绝输入	false alarm 增加，missing 可能减少
较低的 λ	更容易接受输入	false alarm 减少，missing 可能增加
随任务调节的 λ	按应用代价折中	需要 development set 或业务代价信息

不要把分数大小直接等同于答案

Confidence score 是排序与决策的依据，但它本身不是最终答案。只有当阈值、错误代价和应用目标一起确定后，我们才知道某个分数到底应该被解释成 normal 还是 anomaly。

2.3 本章小结

在实际资料中，正常样本和异常样本的 confidence 分布往往会重叠。大多数正常样本分数高，并不代表所有异常样本分数都低。异常检测的困难正在于此：阈值不是从图上随便画一条线，而是在 false alarm 与 missing 之间选择一种任务偏好。

3 Accuracy 的陷阱与两类错误

普通分类任务常用 accuracy 衡量系统表现，但课程特别强调：accuracy 不是异常检测中的好指标。原因并不是 accuracy 的定义错了，而是异常检测资料通常高度不平衡。正常资料很多，异常资料很少；在这种设定下，一个几乎不做事的系统也可能拿到很高 accuracy。

Evaluation

Accuracy is not a good measurement!

A system can have high accuracy, but do nothing.

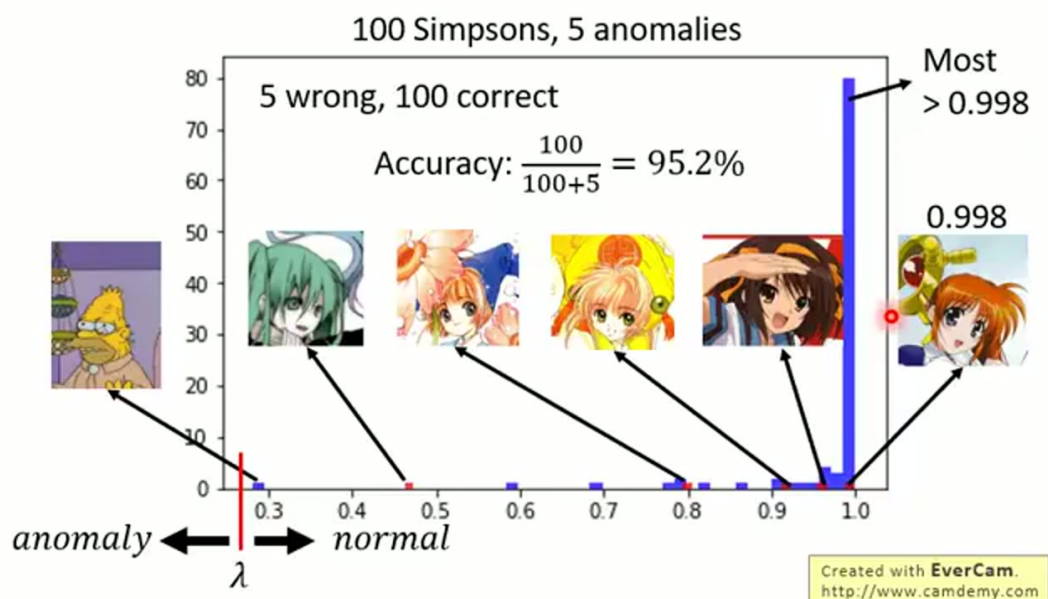


图 4: Accuracy 可能误导异常检测评估: 系统把 5 个异常全放过, 仍可因 100 个正常样本正确而得到 95.2%⁴

3.1 高 accuracy 可能代表系统什么都没做

课程中的例子有 100 张正常图片和 5 张异常图片。假设某个系统把所有样本都判断成 normal, 那么 100 张正常图片都判断对了, 5 张异常图片都判断错了。它的 accuracy 是

$$\frac{100}{100 + 5} = 95.2\%.$$

这个数字看起来很高, 但系统完全没有侦测出任何异常。若用户真正关心的是“异常出现时能不能抓到”, 那么这样的系统几乎没有价值。

这就是异常检测评估的核心陷阱: 当 anomaly 很稀少时, accuracy 会被正常样本数量主导。只要正常样本够多, 模型即使漏掉所有异常, 也可能得到一个漂亮的总准确率。

类别不平衡会掩盖失败

异常检测最怕的不是总错误率看起来高, 而是关键错误被大量正常样本稀释。一个系统如果永远回答 normal, 在许多极端不平衡任务中都能得到高 accuracy, 但它没有完成异常检测的任务。

3.2 两种错误: false alarm 与 missing

为了更细致地描述异常检测表现, 课程区分两类错误:

- **False alarm:** 正常资料被判断成异常。也就是系统误报, 可能造成额外检查、人工复核或用户困扰。

⁴ 视频画面时间区间: 00:08:15–00:08:30。

- **Missing**: 异常资料被判断成正常。也就是系统漏报，可能让真正危险或重要的异常事件没有被处理。

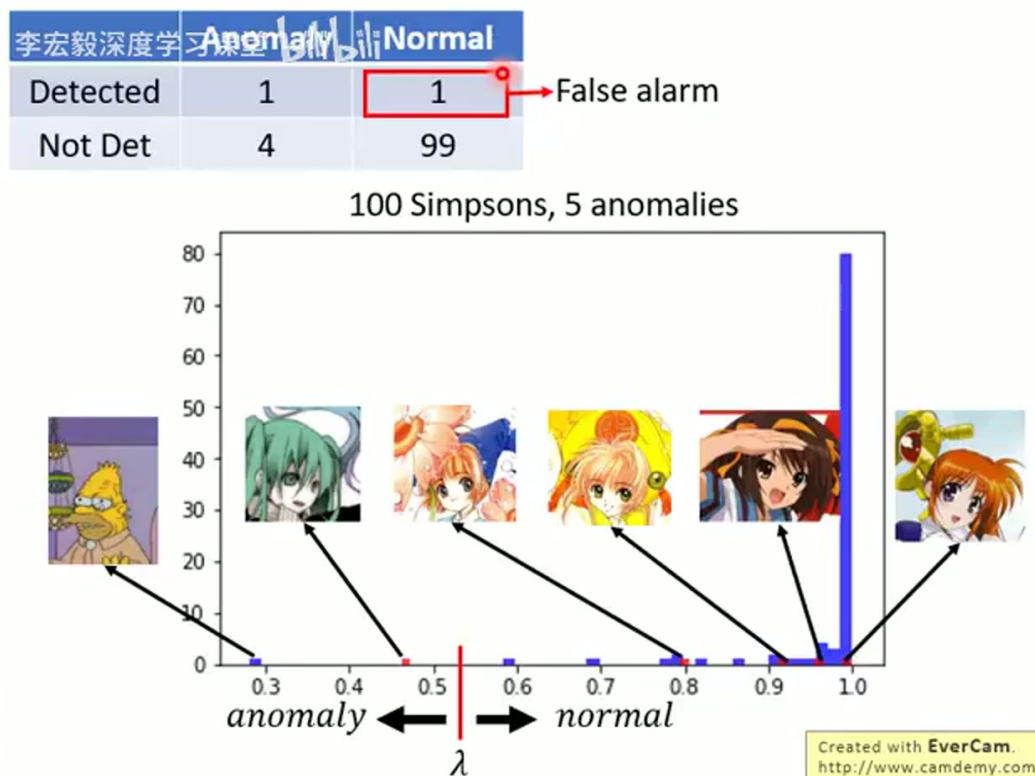


图 5: 同一个阈值会同时决定 false alarm 与 missing: 正常样本被拒绝是误报, 异常样本被接受是漏报⁵

这两类错误没有绝对的谁更严重, 严重程度取决于任务。如果是垃圾邮件过滤, 误把重要邮件判成垃圾邮件可能非常烦; 如果是安检或医疗筛查, 漏掉危险案例可能更严重。异常检测评估不能只问“错了几个”, 还要问“错的是哪一种”。

3.3 用混淆矩阵看错误结构

异常检测的二元决策可以写成混淆矩阵。这里用“Detected”表示系统判断为 anomaly, 用“Not Det”表示系统判断为 normal:

系统输出	真实 anomaly	真实 normal	含义
Detected	正确侦测	false alarm	把输入当成异常
Not Det	missing	正确接受	把输入当成正常

相比单一 accuracy, 混淆矩阵能告诉我们错误是如何分布的: 系统到底是太敏感, 还是太宽松; 问题主要是误报太多, 还是漏报太多。后面讨论 cost table 时, 正是基于这个结构计算不同错误的代价。

3.4 本章小结

异常检测不能只看 accuracy, 因为异常样本通常太少, 导致“全部判断为 normal”的系统也可能得到高分。更有用的评估方式是把错误拆成 false alarm 和 missing: 前者是正常样本被误报成异

⁵ 视频画面时间区间: 00:09:30-00:09:45。

常，后者是异常样本被漏掉成正常。不同应用对两种错误的容忍度不同，因此需要更贴近任务的指标。

4 用 cost table 决定系统偏好

既然 false alarm 和 missing 的重要性会随任务改变，课程进一步引入 cost table。Cost table 的想法很直接：不要假设每一种错误都一样糟，而是给不同预测结果指定不同代价，再计算系统的总成本。总成本较低的系统，才是当前任务偏好下较好的系统。

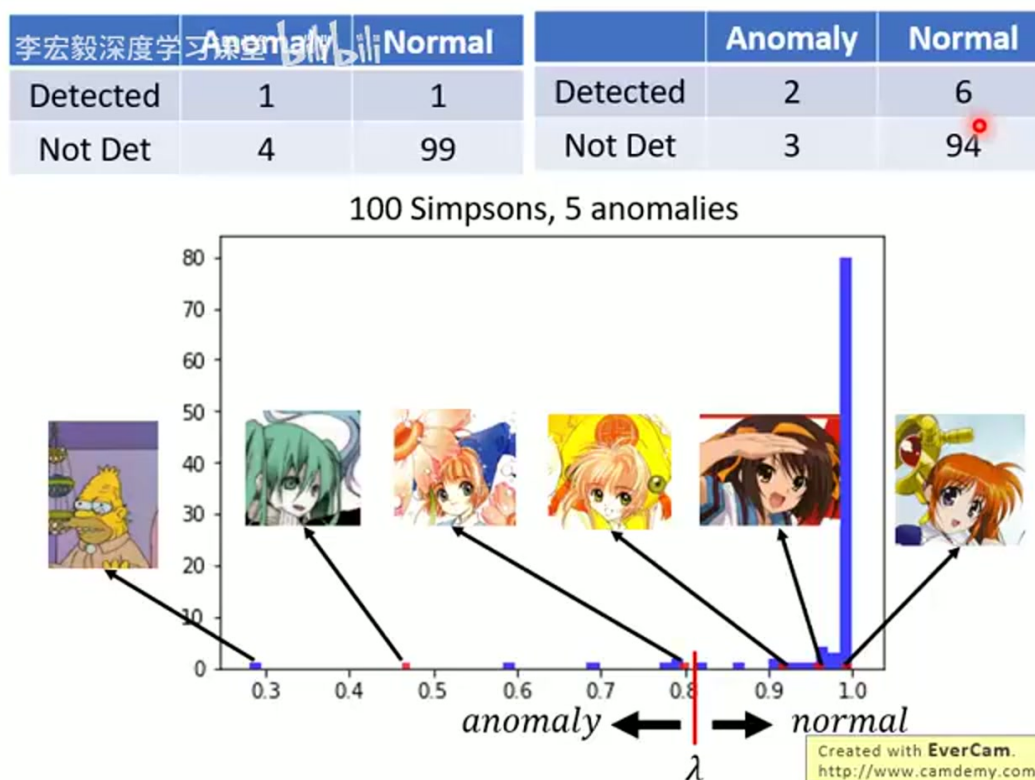


图 6: 两个阈值产生两组混淆矩阵：左边较少 false alarm 但漏掉更多异常，右边抓到更多异常但误报也更多⁶

4.1 两个阈值，对应两种系统行为

课程展示了两个阈值选择。左边系统的混淆矩阵为：

	真实 anomaly	真实 normal
Detected	1	1
Not Det	4	99

它只抓到 1 个异常，漏掉 4 个异常；同时只有 1 个正常样本被误报为异常。这个系统比较保守，不太打扰正常样本，但会漏掉较多异常。

右边系统的混淆矩阵为：

	真实 anomaly	真实 normal
Detected	2	6
Not Det	3	94

⁶ 视频画面时间区间：00:10:45-00:11:00。

它抓到 2 个异常，漏掉 3 个异常；但有 6 个正常样本被误报。这个系统比较敏感，愿意付出更多 false alarm 来换取较少 missing。

4.2 Cost Table A: 误报很贵

第一种 cost table 假设 false alarm 代价很高：把正常样本误判为异常要付出 100 的成本，而漏掉异常只付出 1 的成本。可以写成：

	真实 anomaly	真实 normal
Detected	0	100
Not Det	1	0

于是左边系统的总成本是

$$1 \times 100 + 4 \times 1 = 104,$$

右边系统的总成本是

$$6 \times 100 + 3 \times 1 = 603.$$

在这个代价设定下，左边系统明显更好，因为它大幅减少了误报。

4.3 Cost Table B: 漏报很贵

第二种 cost table 反过来，假设 missing 的代价很高：漏掉异常要付出 100 的成本，而 false alarm 只付出 1 的成本：

	真实 anomaly	真实 normal
Detected	0	1
Not Det	100	0

此时左边系统的总成本变成

$$1 \times 1 + 4 \times 100 = 401,$$

右边系统的总成本变成

$$6 \times 1 + 3 \times 100 = 306.$$

结论完全反过来：右边系统更好。它虽然误报更多，但少漏掉一个异常；在 missing 很贵的任务里，这个交换是值得的。

李宏毅深度学习课程	Anomaly	Normal		Anomaly	Normal
Detected	1	1	Detected	2	6
Not Det	4	99	Not Det	3	94
Cost = 104			Cost = 603		

Cost	Anomaly	Normal	Cost	Anomaly	Normal
Detected	0	100	Detected	0	1
Not Det	1	0	Not Det	100	0
Cost Table A			Cost Table B		

Created with EverCam.
http://www.camdemy.com

图 7: 同两套系统在 Cost Table A 与 Cost Table B 下优劣相反: 评价标准会改变我们选择的模型或阈值⁷

4.4 医学筛查中的直觉

课程用癌症检测解释为什么 cost table 不能脱离任务。假设系统判断一张检查影像是否异常。如果病人其实没有癌症，但系统误判为有癌症，这是 false alarm。它可能造成进一步检查、心理压力和医疗成本，但通常还有后续流程可以纠正。

反过来，如果病人其实有癌症，系统却判断为正常，这就是 missing。它可能导致治疗延迟，后果远比一次额外检查严重。因此在癌症筛查这类任务里，missing 通常比 false alarm 更贵。系统宁愿多发一些警报，也不愿轻易放过真正异常。

指标不是中立的

选择评价指标就是选择任务偏好。Accuracy 假设每个样本、每种错误都同等重要；cost table 则明确告诉模型评估过程：哪些错误更不能接受。异常检测往往需要后者。

4.5 本章小结

Cost table 把异常检测从“数错几个”推进到“错在哪里、代价多大”。同样两个系统，在 false alarm 昂贵的任务里可能选择左边，在 missing 昂贵的任务里可能选择右边。因此，阈值和模型不能脱离应用场景讨论；评估标准本身就是系统设计的一部分。

⁷ 视频画面时间区间：00:12:15–00:12:30。

5 不固定阈值的评价：ranking 与 AUC

前面的讨论都假设系统最终要选择一个阈值 λ ，再把样本分成 normal 和 anomaly。但有些评价指标并不先固定阈值，而是只关心 anomaly score 的排序。课程最后提到的典型例子是 ROC 曲线下面积，也就是 AUC。

李宏毅深度学习	Anomaly	Normal		Anomaly	Normal
Detected	1	1	Detected	2	6
Not Det	4	99	Not Det	3	94
Cost = 104 (输)			Cost = 603		
Cost = 401			Cost = 306 (输)		

Cost	Anomaly	Normal	Cost	Anomaly	Normal
Detected	0	100	Detected	0	1
Not Det	1	0	Not Det	100	0
Cost Table A			Cost Table B		

Some evaluation metrics consider the ranking

For example, Area under ROC curve

Created with EverCam.
<http://www.camdemy.com>

图 8: 某些评价指标只考虑异常分数的排序，例如 ROC 曲线下面积 AUC，因此不需要预先固定一个阈值⁸

5.1 从二元决策到排序

如果系统输出的是 confidence score $c(x)$ ，通常 confidence 越低越可能是异常。为了做排序，可以定义 anomaly score：

$$s(x) = -c(x)$$

或使用任何“分数越高越异常”的等价形式。Ranking metric 关心的是：真正的异常样本是否整体排在正常样本前面。它不急着想问“阈值放在哪里”，而是问“如果从最可疑到最不可疑排序，异常会不会被排到前面”。

这种观点很适合还没有确定部署阈值的阶段。研究者可以先比较不同模型产生的排序质量，再根据实际应用选择工作点。例如医疗筛查可以选择高召回的阈值，内容审核可以选择人工审核量可承受的阈值，设备监控可以选择报警频率不过载的阈值。

5.2 ROC 与 AUC 的含义

ROC 曲线会扫描所有可能阈值，观察不同阈值下 true positive rate 与 false positive rate 的关系。AUC 是 ROC 曲线下面积，可以理解为一种整体排序能力的摘要。直观地说，AUC 越高，表示系统越倾向于把真实 anomaly 排在真实 normal 前面。

⁸ 视频画面时间区间：00:13:45–00:14:00。

更形式化一点，AUC 可以解释成：

$$\Pr(s(x_{\text{anomaly}}) > s(x_{\text{normal}})),$$

也就是随机抽一个异常样本和一个正常样本时，系统把异常样本打得更可疑的概率。这个解释说明了为什么 AUC 不需要固定阈值：它评价的是成对排序，而不是某一个阈值下的二元分类。

5.3 AUC 能回答什么，不能回答什么

AUC 的优点是避免提前选择 λ ，并且比 accuracy 更不容易被单一类别数量支配。它特别适合模型开发早期，用来判断一个异常分数是否有区分能力。

但 AUC 也不是部署答案。真实系统最终往往仍要决定报警阈值、人工复核量、可接受漏报率，以及不同错误的业务代价。两个模型 AUC 接近时，某个模型可能在高召回区域更好，另一个模型可能在低误报区域更好。若应用只关心某个特定工作区间，完整 ROC 曲线、precision-recall 曲线或 cost-sensitive evaluation 会比单一 AUC 更有信息。

开发阶段与部署阶段的指标

开发阶段可以用 AUC 评价排序能力，因为它不依赖某个固定阈值。部署阶段通常还需要根据业务代价选择具体阈值，并报告该阈值下的 false alarm、missing、召回率或成本。

5.4 本章小结

AUC 代表一种不先固定阈值的评价思路：先看模型是否能把异常样本排到正常样本前面。它适合比较 anomaly score 的整体质量，但不能替代最终部署时的阈值选择和代价分析。异常检测评估通常需要同时考虑排序能力、工作点表现和应用代价。

6 总结与延伸

这一讲的主线是：异常检测的困难不只在模型本身，也在评估方式。一个分类器可以给出 confidence score，一个阈值可以把分数变成 normal 或 anomaly，但系统是否“好”，取决于我们如何定义错误、如何看待类别不平衡，以及如何衡量不同错误的代价。

可以把本讲内容压缩成四个判断：

1. Development set 必须模拟 anomaly detection 的测试情境，标签应是“是否来自正常分布”，不是正常类别内部身份。
2. Accuracy 在异常检测中经常误导，因为正常样本数量可能远大于异常样本数量。
3. False alarm 与 missing 是两种不同错误，哪个更严重由应用场景决定。
4. AUC 等 ranking metric 可以在不固定阈值时评价分数质量，但部署时仍要选择具体工作点。

如果把这套逻辑迁移到真实项目，一个较稳妥的异常检测报告不应该只写“accuracy = 95%”。它至少应该说明：测试集中正常与异常比例是多少；阈值如何选择；false alarm 和 missing 分别有多少；如果任务中两种错误代价不同，总成本如何计算；如果还没有固定阈值，则应该报告 AUC 或其他排序指标。

从下一步学习角度看，本讲也埋下了一个更一般的问题：异常检测不是单纯追求更复杂的模型，而是要把模型分数、资料分布、阈值选择和应用代价连在一起。只有评估方式对了，模型改进才有方向；否则模型可能只是把一个漂亮但无用的数字变得更漂亮。